

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON
EXPERIENCE (IOHE)**

ON

**PathWise: AI-Based Academic and Career Recommendation System
for Students**

submitted in partial fulfillment of the requirements for the award of degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by:

JINNY KAPUR (2210990462)
ADITYA SHARMA (2210990072)
SANYAM BHATIA (2210990795)

Supervised By:

Dr. Shikha Tuteja
Associate Professor
Department of Computer & Engineering
Chitkara University, Punjab



**CHITKARA
UNIVERSITY**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

INDEX

S.No.	Title	Page No.
i.	Declaration	(i)
ii.	Abstract	(ii)
iii.	Abstract	(iii)
1.	Introduction	1 – 2
2.	Methodology	3 – 6
3.	Algorithm Design	7 – 9
4.	System Architecture	10 – 12
5.	Technology Stack	13 – 14
6.	System Interface	15 – 23
7.	Results	24
8.	Conclusion & Future Scope	25
9.	References	26
10.	Appendix	27 – 33

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**PathWise: An AI-Based Academic and Career Recommendation System for Students**” in partial fulfillment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Shikha Tuteja. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date:

Jinny Kapur	Aditya Sharma	Sanyam Bhatia
2210990462	2210990072	2210990795

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja

Associate Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to express our sincere gratitude to all those who directly or indirectly contributed to the successful completion of this project and guided us throughout the course of this work. We are thankful for the opportunity to undergo the Industry Oriented Hands-on Experience (IOHE) as a part of the curriculum, which provided us with valuable practical exposure and learning experience.

We would like to extend our heartfelt appreciation to **Dr. Shikha Tuteja** for her constant guidance, valuable suggestions, and continuous support throughout the development of our project, *PathWise*. Her encouragement, constructive feedback, and technical insights greatly helped us in improving the quality of our work and successfully completing the project as well as the associated copyright documentation.

We are also grateful to our faculty members and institution for providing the necessary resources and environment required for the successful execution of this project.

Lastly, we would like to thank our parents, friends, and well-wishers for their continuous motivation, moral support, and encouragement during the entire development process.

Thanking you,

Jinny Kapur	Aditya Sharma	Sanyam Bhatia
2210990462	2210990072	2210990795

ABSTRACT

Choosing an academic and professional path could be challenging. Students and young people are forced to choose one of many possible ways; otherwise, they would squander their time and be dissatisfied with their work later. While many resources help determine which course to take or what job will become popular in the future, there are few options that consider a person's unique set of skills and interests. Therefore, people gather all available advice about their future trajectory and make conclusions based on it. It usually results in an unsatisfying path that could have been better.

PathWise is an AI recommendation platform designed to be personalized and transparent, as opposed to a closed system recommending random choices. Firstly, it obtains information about users' skills, interests, and plans. These pieces of data are introduced by a user into the interface. Next, the system uses a combination of rule-based filtering, cosine similarity and weighted scoring in its evaluation of how well each of these careers matches a user's information using the vast database. Finally, the user obtains sorted recommendations and their corresponding match percentage.

The system goes beyond mere recommendations since it considers a user's current skill gap and suggests the way to improve their competence level to pursue any career of interest. After pinpointing the skillset deficit of an individual, the software produces a customized learning plan containing the necessary skills required for the career path of choice. Additional capabilities include course recommendations, certification programs, and personal projects which are laid out sequentially, pointing toward the next step that needs to be taken. The user can conveniently import his/her resume, and PathWise will extract relevant skills and fill skill gaps automatically.

In its current iteration, PathWise uses modern technologies such as Next.js and TypeScript as the front end, Node.js and Express on the back-end side, and MongoDB as the database management solution with an additional AI service to maintain performance and responsiveness. This architecture makes it highly scalable and easy to update and scale to handle significant amounts of requests while maintaining a smooth user experience. Security measures are also paramount, including JWT tokens, strong password hashing, and OTP verification to secure sensitive information.

With regards to usability, PathWise provides users with interactive dashboards that include bar and radar charts breaking down how every recommendation was generated. Additionally, the platform incorporates anti-cheating functionality for assessment activities, monitoring any tab switches or inactive sessions to ensure that the skills evaluation remains accurate.

While other intelligent recommendation engines fail to justify why certain suggestions were made, PathWise explains each step of the process transparently. In doing so, PathWise clearly demonstrates how the skills and interests of an individual match each career option with numerical ratings. This clarity allows the user to implement the recommendations effectively while building trust in the guidance provided.

Keywords: PathWise, AI-driven recommendation platform, Career Recommendation System, Academic, Guidance, Cosine Similarity, Weighted Scoring, Rule-Based Filtering, Skill Gap Analysis, Personalized Learning Roadmap, Resume Parsing, Explainable AI, Dashboard Visualization, Anti-Cheating System, Next.js, Node.js, MongoDB, TypeScript, JWT Authentication, Microservice Architecture, Career Guidance Platform.

1. INTRODUCTION

1.1 Problem Background

In today's rapidly evolving academic and professional environment, students are required to make important decisions regarding their career paths at an early stage. However, choosing an appropriate career is often a complex and confusing process due to the lack of customized and data-driven approach. While there is plenty of information available on career options, it is typically generic and does not consider individual differences such as skills, interests, and long-term goals.

The majority of existing career guidance services provide general information on job positions, qualification required, or popular sectors. It means that users cannot get any personalized recommendations since there is no connection between a person's current capabilities and requirements of different careers. That is why individuals have to use suggestions from others, carry out independent research, or rely on personal experience.

It is important to mention that another challenge of current platforms consists in the lack of actionability in recommendations. It means that even when there are some recommendations provided, it is hard for people to learn how to become experts in particular career paths. Moreover, existing services usually do not offer a possibility to track one's achievements while working toward becoming successful. In addition to that, nowadays, many recommendation engines are built with the help of complicated algorithms that act as black boxes, and users do not understand how the recommendations were made.

1.2 Problem Statement

- There is a gap in the presence of an intelligent system that would be able:
- To generate personalized recommendations related to different career paths.
- To analyze and compare users' skills and requirements of careers.
- To detect gaps in user's skill set and areas for improvements.
- To explain recommendations in details.
- To provide users with the necessary learning pathways.

The absence of such a system results in a disconnect between a student's potential and the opportunities available, ultimately affecting both academic and professional growth.

1.3 Proposed Approach Overview

In order to tackle these challenges ,a new approach namely PathWise, is suggested as an AI-based platform that allows generating academic and career recommendations based on structured user inputs. The system will collect information related to user skills, interests, and career goals, and use them to generate recommendations via rule-based filtering and similarity-based scoring.

The system evaluates the alignment between user profiles and a number of available career paths by means of calculating the cosine similarity and weighted scoring. Based on this evaluation, the system will be able to provide personalized recommendations on careers, missing skills and learning roadmaps.

The solution is designed to be explainable, ensuring that users can clearly understand why a particular recommendation is made. By integrating data processing, recommendation logic, and visualization into a single platform, PathWise aims to transform career decision-making into a structured and informed process.

2. METHODOLOGY

The development of the PathWise system is based on a structured and iterative approach, where each module—frontend, backend, AI engine, and database—was designed, implemented, and tested independently of others before integration. In essence, the methodology emphasizes transforming unstructured user input into meaningful career recommendations through systematic data processing, similarity-based analysis, and structured output generation.

The overall workflow of the system consists of multiple stages, including data collection, preprocessing, recommendation computation, visualization, and feedback integration.

2.1 Development Approach

Throughout the project development process, the incremental and modular approach was applied so as to facilitate the design, implementation, and maintenance of individual components as well as debugging and component integration process:

- The frontend was developed to handle user interaction and data input.
- The backend was built to perform system operations related to request handling and logic.
- The AI engine was created as a separate micro-service to handle computational tasks
- Finally, the database was developed to store user data, career datasets, and system outputs

This approach ensured efficient debugging, scalability, and smooth integration of components.

2.2 Data Collection

The first step of the methodology involves collecting structured user data through an interactive interface. The system gathers the following inputs:

- User skills
- Interests
- Career goals (optional)

Besides manual entry, resume parsing is available in the system, allowing automatic extraction of relevant data such as skills from a PDF document.

2.3 Data Preprocessing

Before performing any analysis, the collected data is preprocessed to ensure consistency and correctness. The preprocessing steps include:

- Converting text inputs to a standardized format (e.g., lowercase)
- Removal of duplicate entries
- Trimming unnecessary spaces
- Normalization of skill and interest values

This step ensures that the input data is compatible with the predefined career datasets and improves the accuracy of similarity calculations.

2.4 Recommendation Engine

The core of the system is the recommendation engine, which uses a hybrid approach combining rule-based filtering and similarity-based matching.

The process involves:

- Comparison of skills with the required ones for the specified career.
- Matching interest with those associated with particular career paths.
- Identification of missing skills (skill gaps)
- Analysis of overall consistency of career suitability.

The recommendation engine ensures that the system not only suggests careers but also provides meaningful insights into why those recommendations are made.

2.5 Similarity Computation

The system computes the degree of alignment between the user profile and a particular career domain. The procedure includes the following steps:

- User profiles and career requirements are converted into vector representations
- Cosine similarity is calculated between these vectors
- A higher similarity score indicates a stronger alignment

This mathematical approach ensures efficient and accurate comparison between user data and career datasets.

2.6 Weighted Scoring Mechanism

The final score calculation is performed based on the weighted scoring model utilizing the following metrics:

- Similarity Score
- Skill Match Score
- Interest Match Score

The final score is computed as:

$$\textbf{Final Score} = (0.5 \times \textbf{Similarity}) + (0.3 \times \textbf{Skill Match}) + (0.2 \times \textbf{Interest Match})$$

With this formula, both the user similarity and skill and interest match scores are considered when calculating the final score.

2.7 AI Engine Integration

The recommendation logic is built as an integral part of the specified AI engine, which works as a micro-service. The data vector of the user from the backend is forwarded to the AI engine, where the following processes take place:

- Vectorization of input data
- Similarity computation
- Score calculation
- Ranking of career options

Afterwards, the computed data vector is forwarded back from the AI engine to the backend that forwards the information to the frontend. With this approach, it becomes possible to increase the performance of the system, allowing for changing its module independently.

2.8 Visualization of Results

The resulting recommendations will be displayed via visualizations facilitating understanding for the user. The visualization includes:

- Bar charts for career comparison
- Radar charts for skill and interest distribution
- Score breakdown for detailed analysis

Visualization helps users interpret the results more effectively and supports better decision-making.

2.9 Feedback Mechanism

In order to increase the efficiency of the program, a feedback mechanism has been developed, allowing users to provide:

- The rating for the received recommendations.
- Suggestions for future improvements.

Based on such feedback, changes can be applied to the scoring algorithm and database.

2.10 Assessment & Integrity Mechanism

In order to maintain accurate and credible results, an assessment integrity feature is incorporated into the system with anti-cheating functionality. In other words, monitoring the user's actions while conducting an assessment is carried out.

- Warning notifications will be sent in case of multiple violations
- Disqualifying from a test or scoring penalties for recurrent violations will be applied.

Thus, cheating detection helps to maintain the validity of the test results.

3. ALGORITHM DESIGN

In order to create career recommendations for a particular user based on their input, a set of operations is performed using the PathWise algorithm. Data preprocessing, feature matching, and similarity comparison are some of the key stages of generating the output. Overall, the algorithm calculates the level of compatibility between the profile of the user and each possible career domain. It is important to create accurate and understandable suggestions based on the provided data.

The goal of the algorithm is to ensure that recommendations are not only accurate but also explainable, allowing users to understand the reasoning behind each suggestion.

3.1 Input Definition

The algorithm takes the following inputs from the user:

- **Skills (S):** A set of technical or non-technical skills possessed by the user
- **Interests (I):** Areas or domains the user is interested in
- **Career Goals (optional):** Preferred career direction

This is the profile of the user used to compare against different careers.

3.2 Data Processing

Before performing comparisons, the input data undergoes preprocessing:

- Convert all inputs into a standardized format (e.g., lowercase)
- Removing the duplicates from the inputs.
- Normalize values for consistency
- Create a unified feature representation

These stages guarantee proper comparisons of the user data and career datasets.

3.3 Feature Matching

In regard to each career, the following features are retrieved:

- **Set of Required Skills (R)**
- **Related set of Interests (C)**

The algorithm then performs:

- Skill Matching → compares user skills with required skills
- Interest Matching → compares user interests with career interest

This step helps identify both alignment and gaps.

3.4 Similarity Computation

For the purpose of measuring similarity in overall alignment between user profile and career profile, cosine similarity is used.

$$\text{Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- A = User profile vector
- B = Career profile vector

A higher similarity value indicates a stronger match between user and career.

3.5 Score Calculation

The system calculates three key scores:

- **Skill Match Score**
- **Interest Match Score**
- **Similarity Score**

These scores are used to calculate the final score by using following formula:

$$\text{Final Score} = (0.5 \times \text{Similarity}) + (0.3 \times \text{Skill Match}) + (0.2 \times \text{Interest Match})$$

This ensures balanced evaluation while prioritizing overall similarity.

3.7 Ranking Mechanism

- All career domains will be evaluated based on the final score.
- Careers will be ranked in decreasing order of final score
- Top **N recommendations** are selected

This provides users with the most relevant career options.

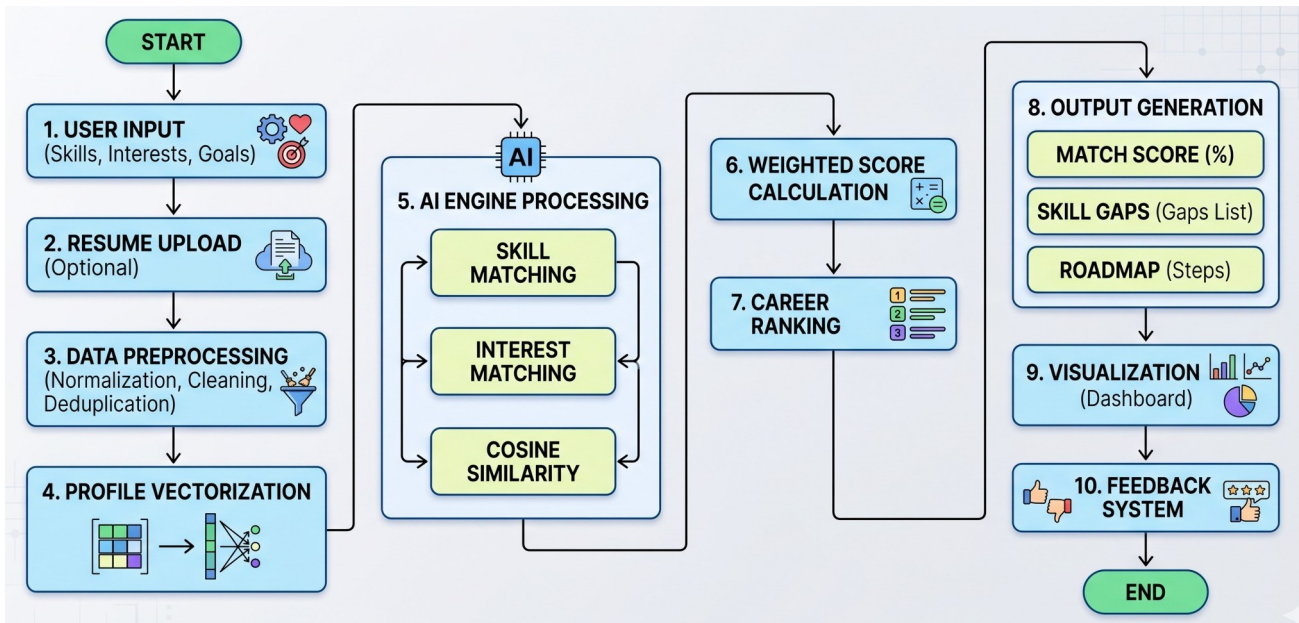
3.8 Output Generation

The system will generate following outputs in a structured manner:

- Match Percentage for each career
- Skill Gap Identification
- Recommended Learning Roadmap
- Detailed Score Breakdown

This makes the recommendation process transparent and actionable.

3.9 Algorithm Flow (Step-by-Step)



4. SYSTEM ARCHITECTURE

PathWise system employs modularity and micro service architecture to ensure efficient execution of all operations. In order to achieve that, the system's architecture will consist of multiple layers each with its distinct responsibilities.

Following are the four main layers in the PathWise architecture:

- Frontend Layer (Presentation Layer)
- Backend Layer (Application Layer)
- AI Engine (Micro service Layer)
- Database Layer (Data Storage)

This way, each layer can be separately developed and independently maintained.

The overall workflow of the system is as follows:

1. User interacts with the frontend interface
2. Frontend sends requests to backend APIs
3. Backend processes the request and communicates with the AI Engine
4. AI Engine performs recommendation logic and returns results
5. Backend sends processed data back to frontend
6. Frontend displays results using visualizations

4.1 Frontend Layer (Presentation Layer)

Frontend layer deals with users' interactions and data representation. The main purposes of this layer include:

- Accepting user input (skills, preferences, ambitions)
- Handling login, sign-up and authentication processes
- Provide resume upload functionality
- Showing user recommendations.
- Visualizing the analysis results.
- Render charts and dashboards

Technologies Used:

- Next.js (React Framework)
- TypeScript
- Tailwind CSS
- Chart libraries (Recharts)

Features:

- Responsive and user-oriented interface
- Real-time data rendering
- Interactive dashboard

4.2 4.2 Backend Layer (Application Layer)

The backend acts as the central system that manages business logic and data flow. Key Responsibilities:

- Managing API requests
- Authenticating users (using JWT)
- Analyzing and processing user data and profile
- Resume uploading and parsing
- Communicate with the AI Engine
- Store and retrieve data from database

Technologies Used:

- Node.js
- Express.js
- TypeScript

Features:

- RESTful API architecture
- Secure data handling
- Efficient request-response management

4.3 AI Engine (Micro-service Layer)

The AI Engine is a separate micro-service responsible for executing recommendation logic.

Key Responsibilities:

- Implementing data vectorization
- Computing cosine similarity
- Calculating weighted score
- Ranking career recommendations
- Highlighting skill gaps
- Generating roadmap suggestions

Key Advantages:

- Independent processing of computational tasks
- Does not interfere with backend performance
- Ability to scale or update separately

With this separation of responsibilities, we achieve the best system efficiency at handling numerous requests at once.

4.4 Database Layer

The database stores all system-related data in a structured format.

Technologies Used: MongoDB (NoSQL Database)

Data Stored:

- User credentials and profiles
- Skills and interests
- Career datasets
- Test attempts and results
- Feedback data

Benefits:

- Flexible schema design
- Effectiveness in handling dynamic data
- Easy integration with Node.js

4.5 Communication Flow

The communication between components follows a structured pipeline:

- Frontend → sends user data to backend
- Backend → validates and processes request
- Backend → sends relevant data to AI Engine
- AI Engine → computes recommendation results
- Backend → receives results and formats response
- Frontend → displays result to the user

This layered communication ensures smooth and efficient system operation.

4.6 System Design Highlights

The proposed architecture provides several important benefits:

- **Micro service-based AI Engine** for scalability
- **Separation of concerns** across different layers
- **REST API communication** for high level of flexibility
- **Use of TypeScript** for type safety and maintainability
- **Efficient data flow** between the components
- Support for future expansion and integration

4.7 Advantages of Architecture

- Improves system scalability
- Allows independent updates of modules
- Enhances performance by isolating heavy computation
- Ensures clean and maintainable code structure
- Supports integration with future AI/ML enhancements

5. TECHNOLOGY STACK

PathWise system features modern full-stack architecture comprising frontend, backend, database, and AI components. The technologies used will allow developing a scalable, effective, and performant solution.

5.1 Frontend Technologies

The frontend is responsible for user interaction and visualization.

- **Next.js (React Framework):** Used to build a fast, scalable, and server-rendered user interface.
- **TypeScript:** Ensures type safety, reduces runtime errors, and improves code maintainability.
- **Tailwind CSS:** Used for designing a responsive and clean user interface.
- **Chart Libraries (Recharts or similar):** Used for data visualization such as bar charts, radar charts, and score analysis.

5.2 Backend Technologies

Backend is responsible for business logic, API management, and communication between components.

- **Node.js:** Provides a runtime environment for building scalable server-side applications.
- **Express.js:** Used to create RESTful APIs and manage request-response handling.
- **TypeScript:** Improves code structure and reliability in backend development.

5.3 Database

- **MongoDB:** A NoSQL database that is used for storing user profiles, skill set, career dataset and other relevant system information.
- **Mongoose (ODM):** Used for schema definition and database interaction.

5.4 AI Engine Technologies

The recommendation system is implemented as an independent microservice.

Node.js + TypeScript: Used to build the AI engine that processes recommendations.

Implements:

- Cosine similarity calculations
- Skill and interest matching
- Weighted scoring mechanism

5.5 Authentication & Security

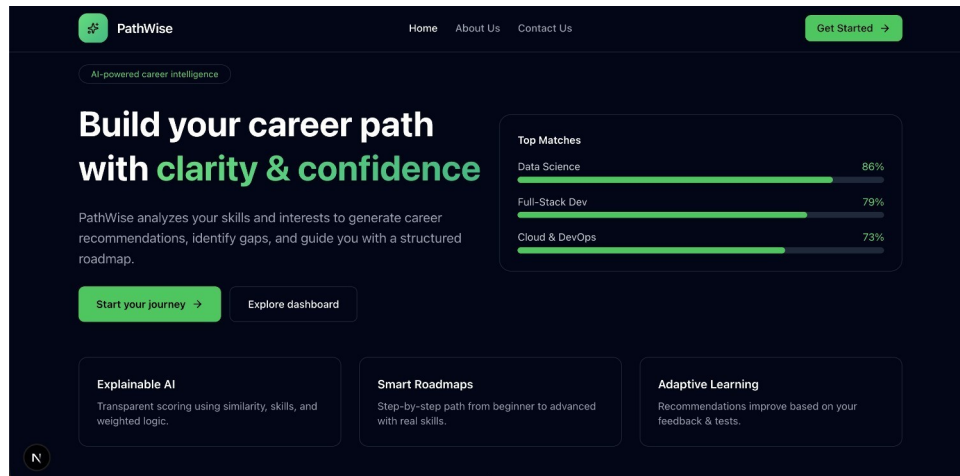
- **JWT (JSON Web Tokens):** Used for secure user authentication and session management.
- **Bcrypt:** Used for hashing user passwords to ensure security.
- **OTP Verification (via Email):** Used for user verification and password reset functionality.

5.6 Additional Tools & Libraries

- **Multer:** Handles file uploads (resume upload functionality).
- **PDF Parser:** Extracts skills and information from uploaded resumes.
- **REST APIs:** Enable communication between frontend, backend, and AI engine.

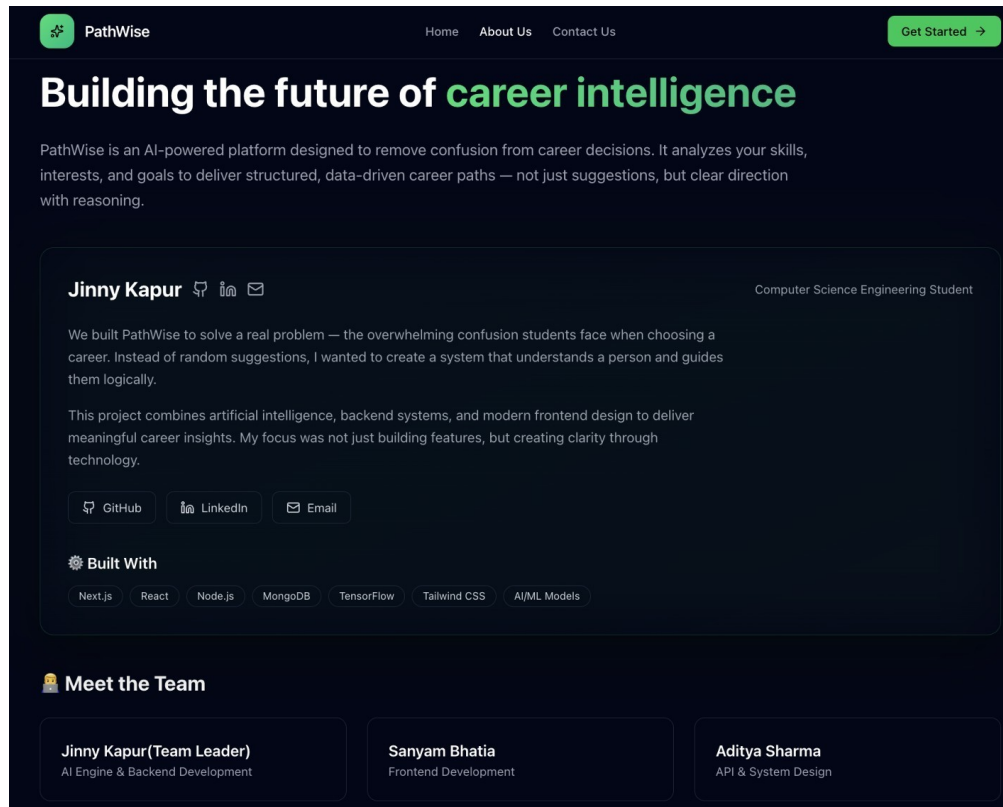
6. SYSTEM INTERFACE

6.1 Home Page



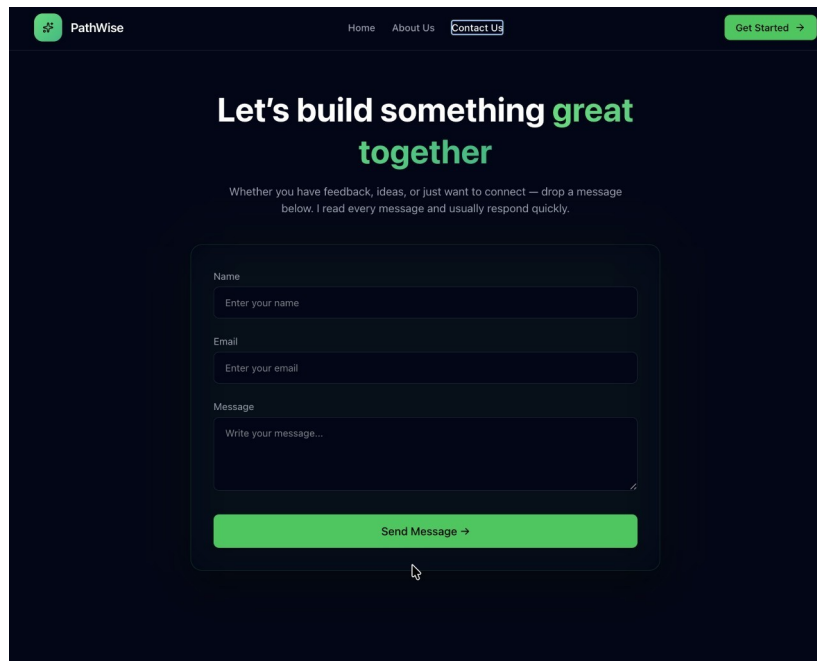
6.2 About Page

Here, users can read about the purpose and functionality of the system, and learn about the problem domain. Additionally, there is brief project description, tech stack and info about the development team included.



6.3 About Page

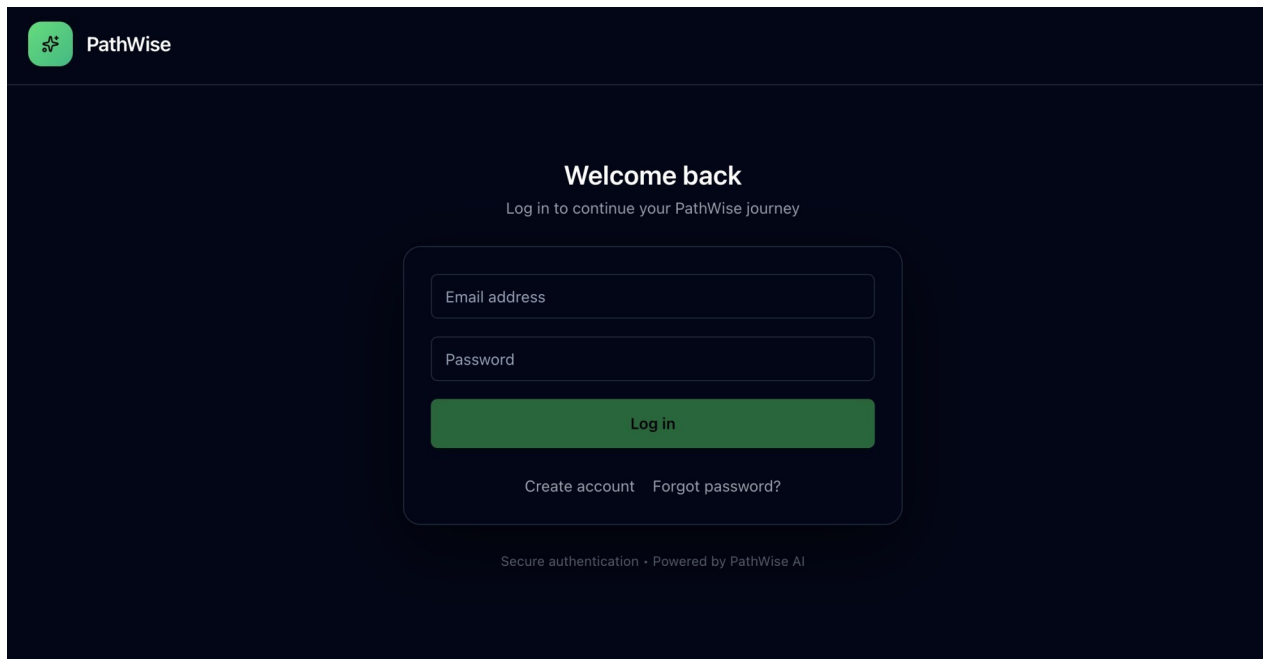
This section explains the purpose of the system and the problem it aims to solve. It provides an overview of the project, technologies used, and introduces the development team.



The screenshot displays the 'About' page of the PathWise application. The header includes the PathWise logo, navigation links for 'Home', 'About Us', and 'Contact Us', and a 'Get Started' button. The main heading reads 'Let's build something great together'. Below this, a subheading states: 'Whether you have feedback, ideas, or just want to connect — drop a message below. I read every message and usually respond quickly.' The contact form consists of three input fields: 'Name' (placeholder: 'Enter your name'), 'Email' (placeholder: 'Enter your email'), and 'Message' (placeholder: 'Write your message...'). A green 'Send Message ->' button is positioned at the bottom of the form.

6.4 Login Page

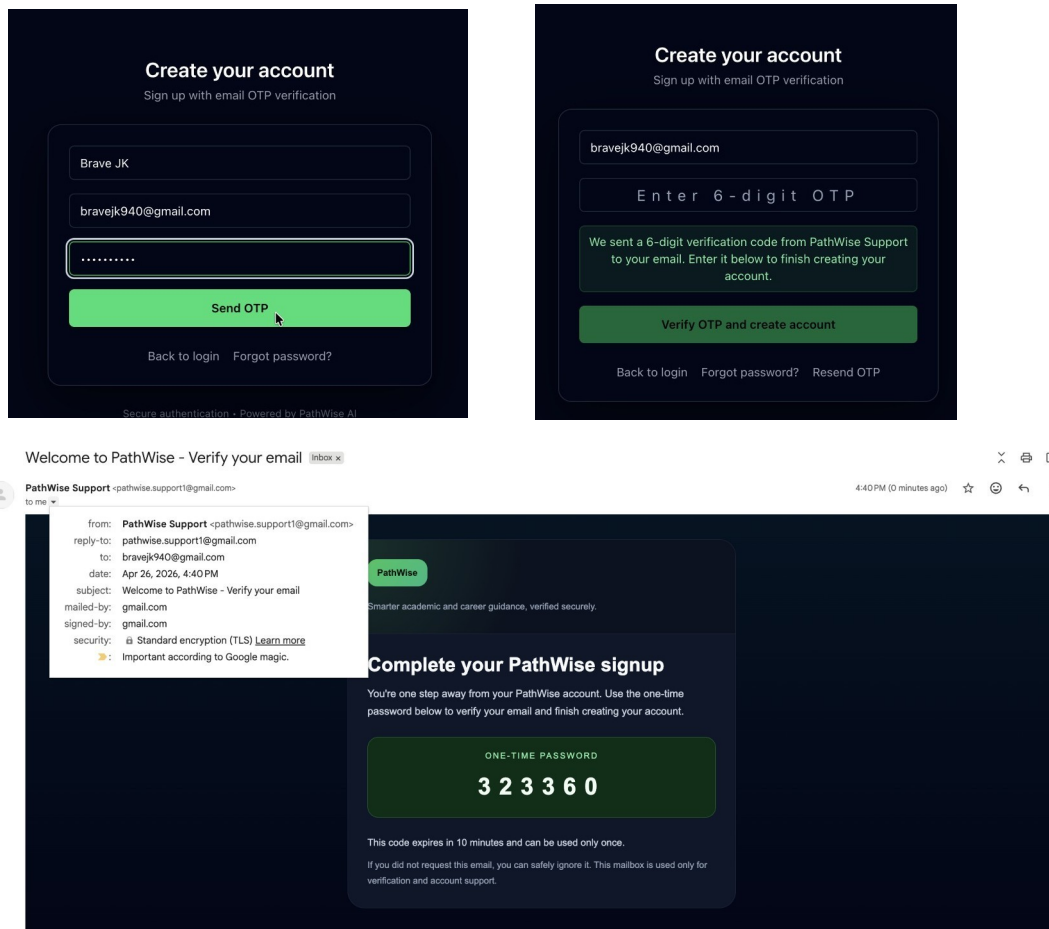
This page enables existing users to securely log into the system using email and password. It ensures authentication before accessing personalized features.



The screenshot shows the login page of the PathWise application. The header features the PathWise logo. The main heading is 'Welcome back', followed by the subheading 'Log in to continue your PathWise journey'. The login form contains two input fields: 'Email address' and 'Password'. A green 'Log in' button is located below these fields. At the bottom of the form, there are links for 'Create account' and 'Forgot password?'. The footer text reads 'Secure authentication • Powered by PathWise AI'.

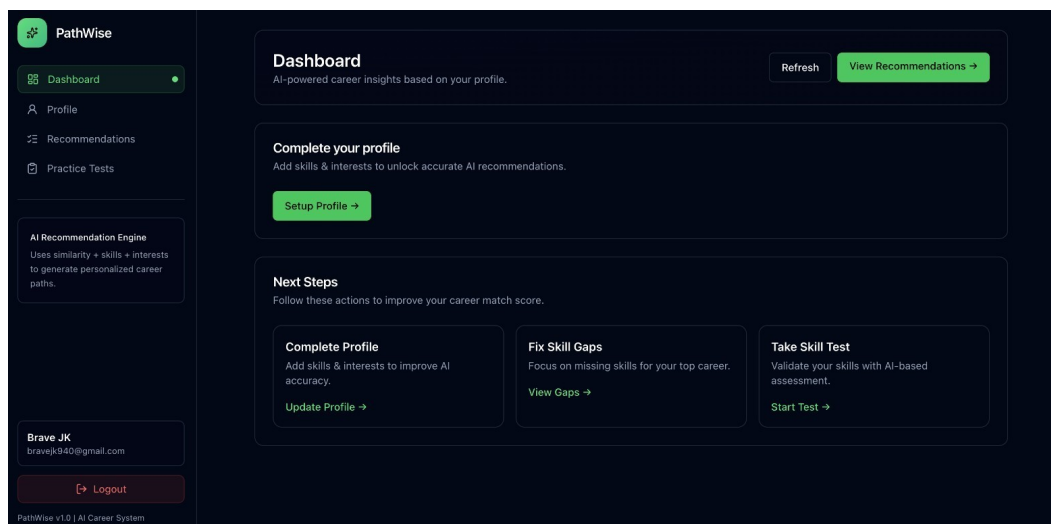
6.5 Signup & OTP Verification

This section demonstrates the account creation process using OTP-based email verification. It ensures secure user registration and prevents unauthorized access.



6.6 Dashboard

The dashboard serves as the central interface where users can monitor their profile status, recommendations, and next steps. It also provides quick navigation to all major features of the system.



6.7 Profile Setup & Resume Parsing

Users can input or automatically extract skills and interests using resume parsing. This data is essential for generating accurate and personalized career recommendation.

The screenshot shows the 'Profile setup' page in the PathWise application. The left sidebar contains navigation links: Dashboard, Profile (active), Recommendations, and Practice Tests. Below these is a section for the 'AI Recommendation Engine' and a user profile for 'Brave JK' with a 'Logout' button. The main content area is titled 'Profile setup' and includes a sub-header 'Resume text extraction' with instructions to upload a resume. A file 'UG-Resume-with-Projects-1.pdf' has been uploaded. Below this, an 'AI Insights' section states that 22 skills and 9 interests were detected. These are displayed in two columns: 'Extracted Skills' (Python, MATLAB, Java, C, C++, JavaScript, TypeScript, HTML, CSS, Django, Flask, Docker, Git, Machine Learning, Scikit-learn, TensorFlow, PyTorch, NLP, Algorithms, Pandas) and 'Extracted Interests' (Research, Security, Systems, Teaching, Data, AI, Cloud, Robotics, Finance). A 'Re-apply extracted data' button is at the bottom right.

This screenshot shows the 'Skills' and 'Interests' selection section of the PathWise application. The 'Skills' section allows users to select their current skills from a list of 22 extracted skills. A search bar is provided, and the selected skills are displayed as a row of tags: Python, MATLAB, Java, C, C++, JavaScript, TypeScript, HTML, CSS, Django, Flask, Docker, Git, Machine Learning, Scikit-learn, TensorFlow, PyTorch, NLP, Algorithms, Pandas, NumPy, and Monitoring. The 'Interests' section allows users to select their interests from a list of 9 extracted interests. A search bar is provided, and the selected interests are displayed as a row of tags: Research, Security, Systems, Teaching, Data, AI, Cloud, Robotics, and Finance.

This screenshot shows the 'Career Direction' section of the PathWise application. It includes a text input field for users to specify their career goals, with an example: 'e.g., Become an AI engineer, build SaaS products...'. Below the input field, a 'Save Profile' button is visible. The page also displays the 'Interests' section from the previous screenshot, showing the selected interests: Research, Security, Systems, Teaching, Data, AI, Cloud, Robotics, and Finance.

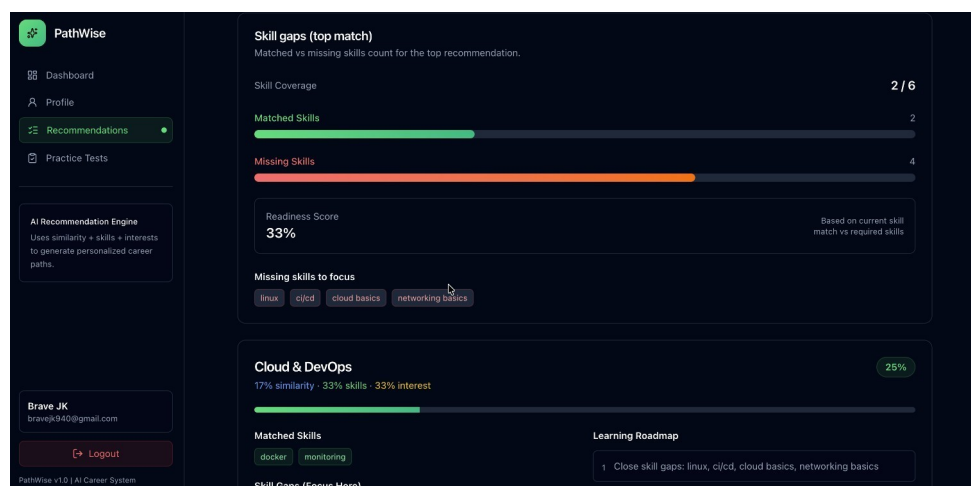
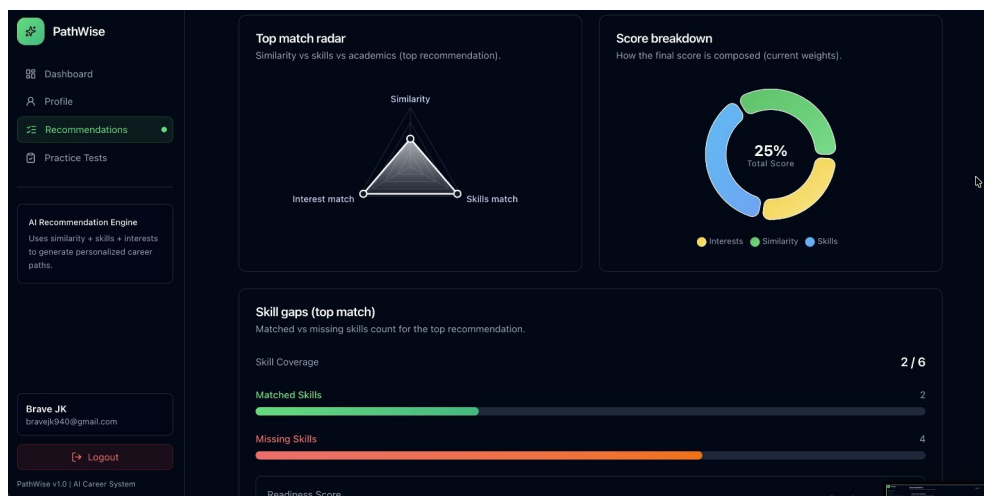
6.8 Recommendations Overview

This section displays career recommendations in the form of comparison graphs. Each career is ranked based on the match percentage calculated by the system.



6.9 Detailed Analysis

Provides an in-depth analysis including radar charts, score breakdown, and skill gap identification. It also helps users understand how well they align with recommended careers.



6.10 Career Recommendation Detail + Roadmap + Feedback

Displays detailed information for each career, including matched skills, missing skills, together with a structured learning roadmap. Users can also provide feedback to improve recommendations.

The screenshot shows the PathWise interface for the 'Cloud & DevOps' career recommendation. The left sidebar contains navigation links: Dashboard, Profile, Recommendations (active), and Practice Tests. Below these is a section for the 'AI Recommendation Engine' and a user profile for 'Brave JK' with a 'Logout' button. The main content area displays the 'Cloud & DevOps' career with a progress bar at 25%. It includes 'Matched Skills' (docker, monitoring), 'Skill Gaps (Focus Here)' (linux, ci/cd, cloud basics, networking basics), and 'Matched Interests' (systems). A 'Learning Roadmap' lists four steps: 1. Close skill gaps: linux, ci/cd, cloud basics, networking basics; 2. Take courses: Docker & Kubernetes - AWS Cloud Practitioner Essentials; 3. Earn certifications: AWS Certified Cloud Practitioner; 4. Build 2-3 relevant projects aligned to this domain and document them well. A 'View full roadmap' link is provided. A 'Feedback' section includes a rating of 5 and a text input for suggestions, with a 'Submit' button.

PathWise

Dashboard
Profile
Recommendations
Practice Tests

AI Recommendation Engine
Uses similarity + skills + interests to generate personalized career paths.

Brave JK
bravejk940@gmail.com
Logout

PathWise v1.0 | AI Career System

Cloud & DevOps
17% similarity · 33% skills · 33% interest

Matched Skills
docker monitoring

Skill Gaps (Focus Here)
linux ci/cd cloud basics networking basics

Matched Interests
systems

Learning Roadmap

- 1 Close skill gaps: linux, ci/cd, cloud basics, networking basics
- 2 Take courses: Docker & Kubernetes - AWS Cloud Practitioner Essentials
- 3 Earn certifications: AWS Certified Cloud Practitioner
- 4 Build 2-3 relevant projects aligned to this domain and document them well.

[View full roadmap →](#)

Feedback

Rating 5

Improve suggestions...

Submit

The screenshot shows the PathWise interface for the 'Cybersecurity' career recommendation. The left sidebar is identical to the first screenshot. The main content area displays the 'Cybersecurity' career with a progress bar at 18%. It includes 'Matched Skills' (networking basics, linux, security fundamentals, scripting, incident response), 'Skill Gaps (Focus Here)' (networking basics, linux, security fundamentals, scripting, incident response), and 'Matched Interests' (security, systems). A 'Learning Roadmap' lists four steps: 1. Close skill gaps: networking basics, linux, security fundamentals, scripting, incident response; 2. Take courses: Introduction to Cyber Security - Linux Fundamentals; 3. Earn certifications: CompTIA Security+; 4. Build 2-3 relevant projects aligned to this domain and document them well. A 'View full roadmap' link is provided. A 'Feedback' section includes a rating of 5 and a text input for suggestions, with a 'Submit' button.

PathWise

Dashboard
Profile
Recommendations
Practice Tests

AI Recommendation Engine
Uses similarity + skills + interests to generate personalized career paths.

Brave JK
bravejk940@gmail.com
Logout

PathWise v1.0 | AI Career System

Cybersecurity
9% similarity · 0% skills · 67% interest

Matched Skills
networking basics linux security fundamentals scripting incident response

Skill Gaps (Focus Here)
networking basics linux security fundamentals scripting incident response

Matched Interests
security systems

Learning Roadmap

- 1 Close skill gaps: networking basics, linux, security fundamentals, scripting, incident response
- 2 Take courses: Introduction to Cyber Security - Linux Fundamentals
- 3 Earn certifications: CompTIA Security+
- 4 Build 2-3 relevant projects aligned to this domain and document them well.

[View full roadmap →](#)

Feedback

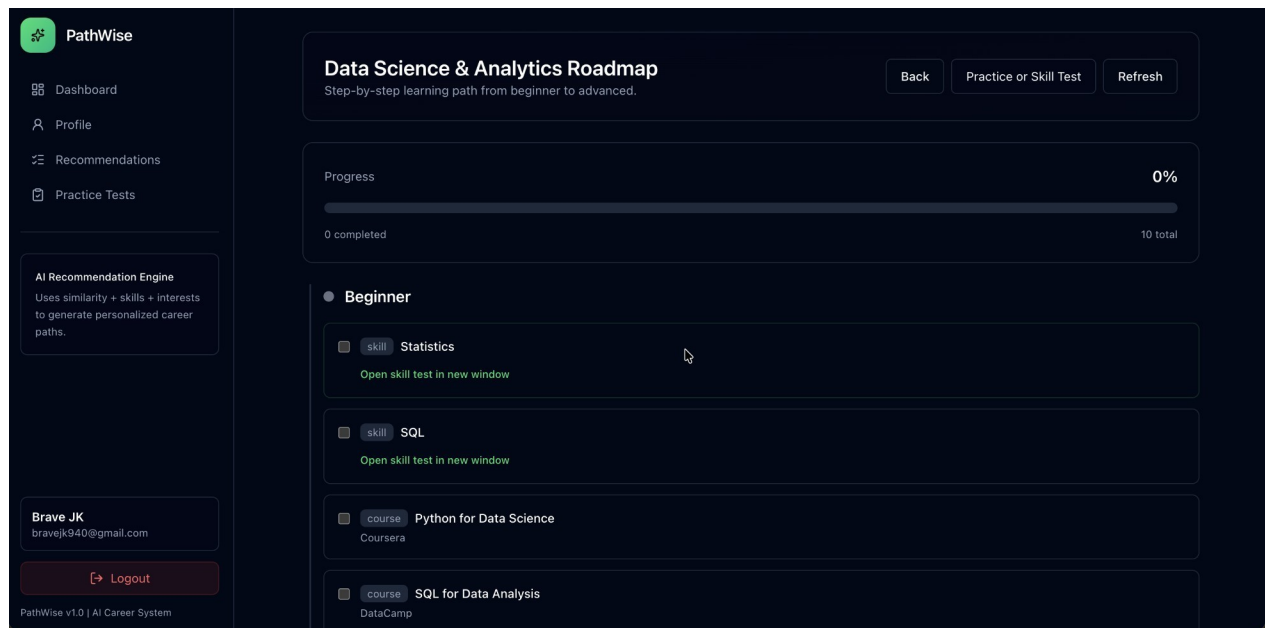
Rating 5

Improve suggestions...

Submit

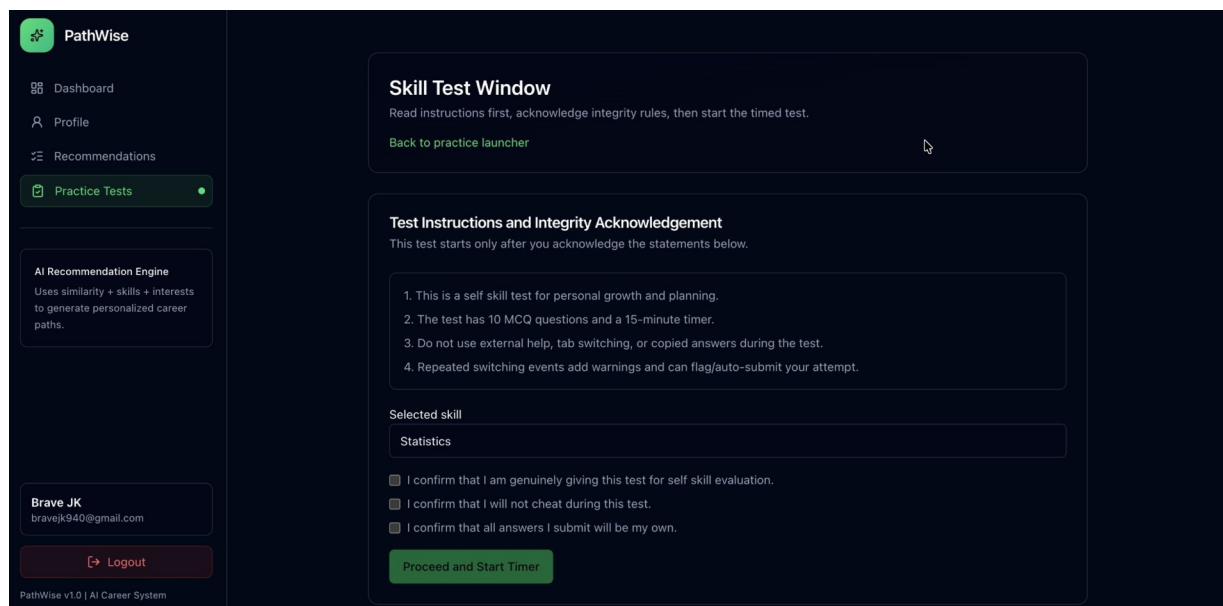
6.11 Roadmap Page

This section shows the recommended learning path with steps to become a successful professional starting from novice level. It also helps users to track progress and follow a structured plan.



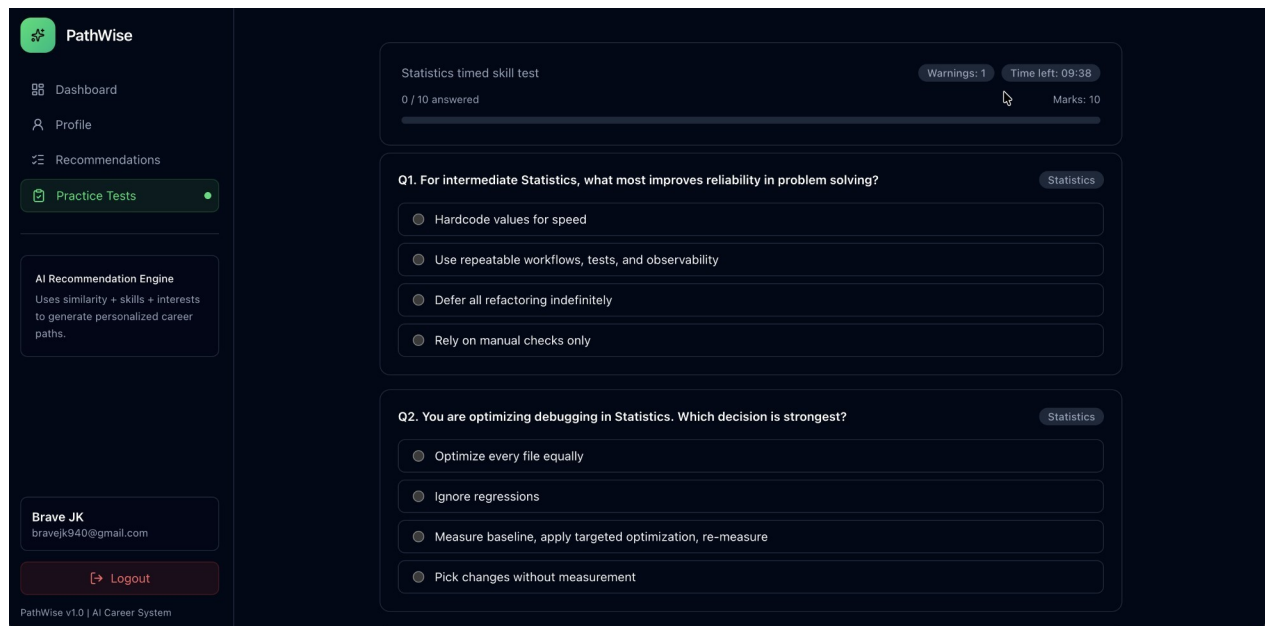
6.12 Practice Test – Instructions & Integrity

This section introduces the test module and displays rules and integrity guidelines to ensure fairness. It ensures users to understand the test conditions before starting.



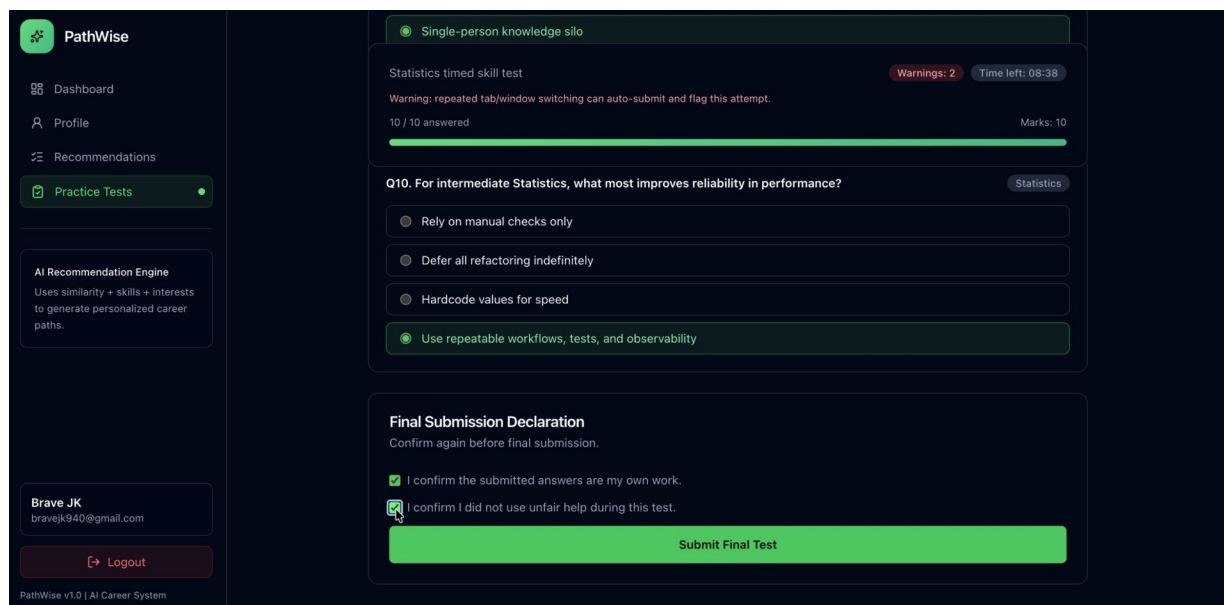
6.13 Test Interface

Displays the MCQ-based assessment interface where users answer questions to evaluate their skills. It includes timers and progress tracking.



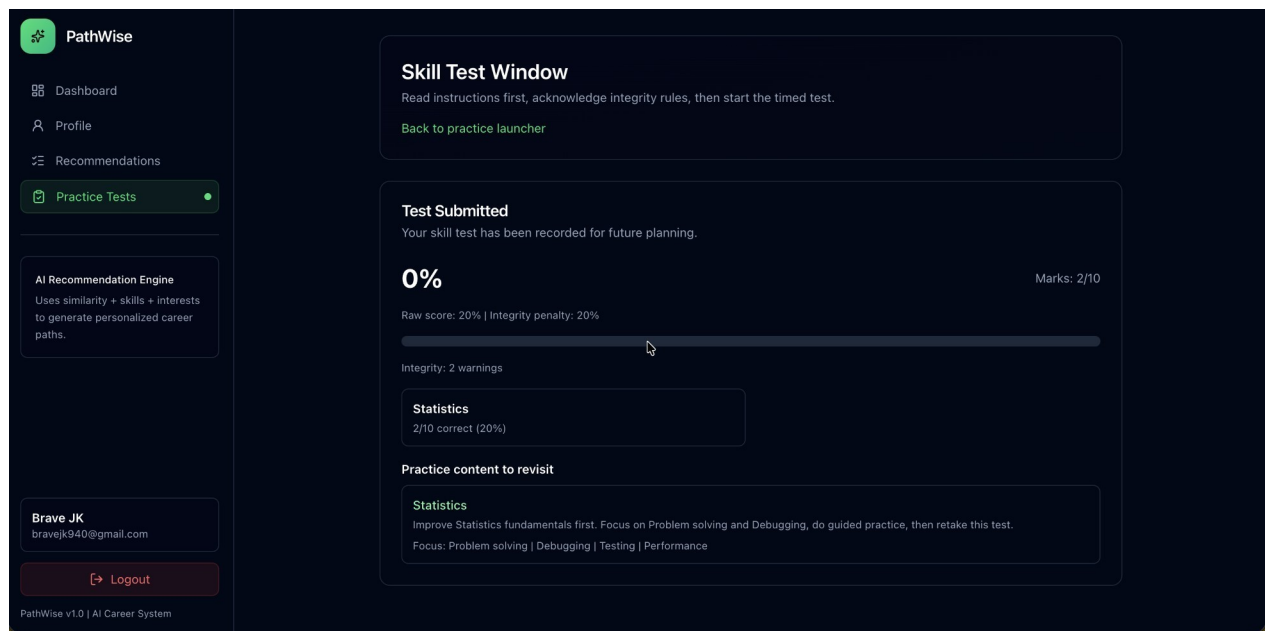
6.14 Anti-Cheat Detection

The system detects suspicious behavior such as tab switching during tests and generates warnings. Users cannot switch between browser tabs while testing is still active.



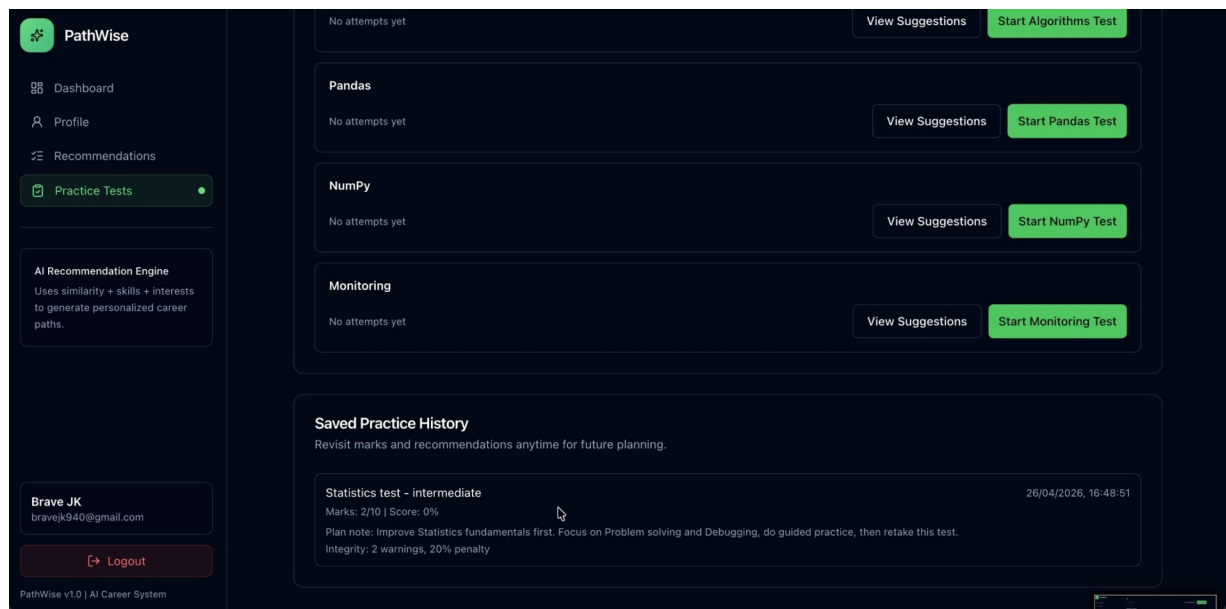
6.15 Test Completion & Submission

Shows final test submission, score calculation, and integrity evaluation. It provides feedback on user performance and penalties (if any).



6.16 Practice Test Dashboard

Displays a list of available skill-based tests and previous attempts. Users can start new tests or review past performance.



7. RESULTS

The PathWise system was shown to have the capability of recommending personalized career opportunities based on user's skills, interests, and career aspirations. Recommendation engine was using cosine similarity and weighted scoring to analyze user profiles and create a list of recommended career domains together with match percentage

The system was able to perform the following tasks:

- Receiving appropriate career domains recommendations, such as Data Science, Cloud & DevOps, Cybersecurity, and Full-Stack Development
- Detection of skill gaps between user profiles and career domains
- Generating personalized learning roadmaps including courses, certifications, and project suggestions
- Display recommendation insights using charts and dashboards

Resume parsing module proved itself effective in extracting skills and interests from uploaded resumes. The subsystem of authentication was working safely and efficiently, using OTP validation together with JWT authentication. Besides, the system of assessments successfully conducted multiple-choice questions tests and tab switching detection.

In conclusion, the goal of providing users with intelligent and structured career recommendations through a scalable full-stack solution was successfully reached.

8. CONCLUSION & FUTURE SCOPE

PathWise has been created with the intent of addressing the problem of lack of personalized and structured career guidance for students and young professionals. The system uses similarity-based approach in combination with user's skills, interests, and career aspirations to give out smart and clear career recommendation.

As a result of work on the project, the following main goals were achieved:

- Providing personalized career recommendations based on user profiles' analysis
- Revealing skill gaps between users and their target careers
- Creation of learning path for future success
- Parsing of resumes to automatically find out missing skills and interests
- Dashboard visualization using charts and score analysis
- Secure authentication using JWT and OTP verification
- Practice test module with anti-cheating and tab-switch detection mechanisms

Apart from career recommendations, the platform provides an insight into what actions should be taken to reach desired career domain. Usage of cosine similarity and weighting makes recommendation algorithms fully interpretable.

However, with the help of micro-services, the AI engine is decoupled from backend operations, increasing system performance. Although the current system produces well-recommended advice, there exists a possibility of improvement in the system's functionality. These may be:

- Integration of advanced machine learning and deep learning models
- Use of Natural Language Processing (NLP) for better resume parsing
- Analysis of the real time job market and other trends.
- AI-based interview preparation and mock assessments
- Integration with online learning platforms for automatic course suggestions for users.
- Real-time tracking of the user's progress and learning process optimization.
- AI-based proctoring features such as webcam monitoring
- Expansion of career datasets to support additional domains and professions

Overall, PathWise serves as a comprehensive decision-support platform that bridges the gap between user potential and career opportunities through intelligent analysis, explainable recommendations, and structured learning guidance.

9. REFERENCES

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
3. React Documentation
<https://react.dev/>
4. Next.js Documentation
<https://nextjs.org/docs>
5. Node.js Documentation
<https://nodejs.org/en/docs>
6. Express.js Documentation
<https://expressjs.com/>
7. MongoDB Documentation
<https://www.mongodb.com/docs/>
8. TypeScript Documentation
<https://www.typescriptlang.org/docs/>
9. Tailwind CSS Documentation
<https://tailwindcss.com/docs>
10. JSON Web Token (JWT) Documentation
<https://jwt.io/introduction>
11. Recharts Documentation
<https://recharts.github.io/?p=/en-US>
12. Cosine Similarity Reference
<https://www.geeksforgeeks.org/cosine-similarity/>
13. Multer Middleware Documentation
<https://github.com/expressjs/multer>
14. Mongoose Documentation
<https://mongoosejs.com/docs/>
15. OWASP Authentication Best Practices
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

10. APPENDIX

The appendix section contains supplementary technical material related to the implementation and structure of the PathWise system. These details provide additional insights into the project architecture, development workflow, and execution environment.

10.1 Folder Structure

The PathWise project follows a modular folder organization to maintain scalability, readability, and separation of concerns. The system is divided into frontend, backend, and AI engine modules.

```
PathWise/
├── frontend/                                # User Interface (Next.js)
│   ├── src/
│   │   ├── app/                            # Application pages & routing
│   │   ├── components/                    # Reusable UI components
│   │   ├── data/                          # Static datasets
│   │   ├── lib/                           # Utility & API functions
│   │   └── public/                         # Static assets
│   └── public/
├── backend/                                # Main Backend Server
│   ├── src/
│   │   ├── routes/                        # API route handlers
│   │   ├── models/                       # MongoDB schemas/models
│   │   ├── middleware/                   # Authentication & error handling
│   │   ├── services/                     # Business logic & integrations
│   │   ├── utils/                        # Helper utilities
│   │   ├── config/                       # Environment configuration
│   │   └── scripts/                       # Seeder scripts
│   └── public/
├── ai-engine/                              # Recommendation Microservice
│   ├── src/
│   │   └── recommendation/
│   │       ├── vectorize.ts               # Profile vectorization
│   │       ├── cosine.ts                 # Similarity calculation
│   │       ├── recommend.ts               # Recommendation logic
│   │       └── normalize.ts               # Data normalization
│   └── public/
├── database/
│   ├── seed/                              # Career datasets
│   └── docker-compose.yml                 # Database container setup
├── scripts/                               # Utility scripts
├── package.json
└── README.md
```

10.2 UML Diagram

The Use Case Diagram illustrates the interaction between different actors and the PathWise system functionalities.

Actors Included

- User (Student / Job Seeker)
- Admin
- AI Engine (Recommendation Service)
- Email / SMS Service

Major User Functionalities

- User Registration and Login
- OTP Verification
- Profile Management
- Resume Upload and Skill Extraction
- Career Recommendation Generation
- Skill Gap Analysis and Roadmap Viewing
- Practice Test and Assessment
- Viewing Test Results
- Feedback Submission

Admin Functionalities

- Manage Users
- Manage Careers and Roadmaps
- Manage Tests (MCQ Bank)
- View Feedback and Reports

Recommendation Workflow

The diagram also represents the internal recommendation process:

- Sending profile data
- Similarity computation
- Recommendation generation
- Learning roadmap generation

Assessment Workflow

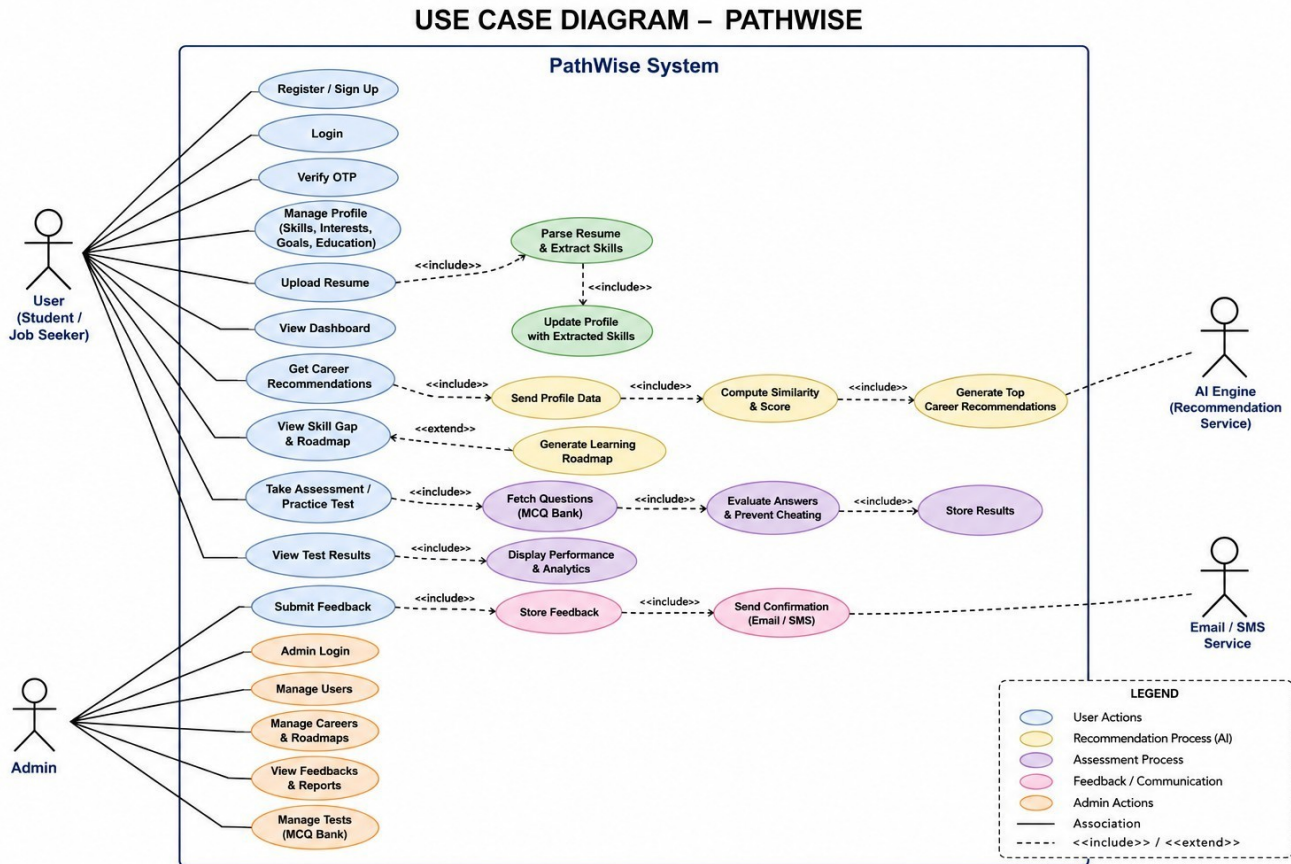
The assessment module includes:

- Fetching MCQ questions
- Evaluating answers
- Anti-cheating and tab-switch monitoring
- Result storage and analytics

Communication Services

The system integrates external services for:

- AI-based recommendation processing
- Email/OTP confirmation and notifications



10.3 Terminal Output

The terminal output represents the successful operation of the PathWise system during its development phase. It uses a concurrent execution model to initialize the frontend, backend, and AI engine in parallel.

Key Observations from Terminal Output

- The frontend server is successfully running using Next.js at:
 - <http://localhost:3000>
- The backend server is successfully initialized at:
 - <http://localhost:5050>
- The AI recommendation engine microservice is actively running at:
 - <http://localhost:7070>

- Environment variables were successfully loaded from the .env configuration files.
- The system uses:
 - ts-node-dev for hot-reload development
 - TypeScript runtime execution
 - Concurrent service execution using the concurrently package
- The successful startup logs indicate:
 - Proper communication between frontend, backend, and AI engine
 - Correct initialization of APIs and recommendation services
 - Stable runtime execution of all modules

The terminal logs validate:

- Successful project compilation and execution
- Proper microservice communication
- Backend and AI engine initialization
- Frontend deployment status
- Runtime debugging and development workflow

```
jinnypur@Jinnys-MacBook-Air PathWise % npm run dev

> dev
> concurrently -n frontend,backend,ai -c cyan,green,magenta "npm --prefix frontend run dev" "npm --prefix backend run dev" "npm --prefix ai-engine run dev"

[backend]
[backend] > backend@1.0.0 dev
[backend] > ts-node-dev --respawn --transpile-only src/index.ts
[backend]
[ai]
[ai] > ai-engine@1.0.0 dev
[ai] > ts-node-dev --respawn --transpile-only src/index.ts
[ai]
[frontend]
[frontend] > frontend@0.1.0 dev
[frontend] > next dev
[frontend]
[backend] [INFO] 11:25:24 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[ai] [INFO] 11:25:24 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[backend] ◊ injected env (20) from .env // tip: % enable debugging { debug: true }
[ai] ◊ injected env (3) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
[ai] [ai-engine] listening on http://localhost:7070
[frontend] ⚠ Warning: Next.js inferred your workspace root, but it may not be correct.
[frontend] We detected multiple lockfiles and selected the directory of /Users/jinnypur/Desktop/PathWise/package-lock.json as the root directory.
[frontend] To silence this warning, set `turbo.root` in your Next.js config, or consider removing one of the lockfiles if it's not needed.
[frontend] See https://nextjs.org/docs/app/api-reference/config/next-config-js/turbo#root-directory for more information.
[frontend] Detected additional lockfiles:
[frontend]   * /Users/jinnypur/Desktop/PathWise/frontend/package-lock.json
[frontend]
[frontend] ▲ Next.js 16.1.7 (Turbo)
[frontend] - Local:      http://localhost:3000
[frontend] - Network:    http://192.168.1.13:3000
[frontend] - Environments: .env
[frontend]
[frontend] ✓ Starting...
[backend] [backend] listening on http://localhost:5050
[frontend] ✓ Ready in 2.4s
```

10.4 API ENDPOINTS

The PathWise system uses RESTful APIs to enable communication between the frontend, backend, database, and AI recommendation engine. These APIs handle authentication, recommendation generation, resume processing, assessments, feedback, and administrative operations.

Authentication APIs

Method	Endpoint	Description
POST	/api/auth/signup	Register a new user
POST	/api/auth/login	Authenticate user login
POST	/api/auth/send-otp	Send OTP for email verification
POST	/api/auth/verify-otp	Verify OTP entered by the user
POST	/api/auth/logout	Logout authenticated user
POST	/api/auth/reset-password	Reset user password

Profile Management APIs

Method	Endpoint	Description
GET	/api/profile	Retrieve user profile
POST	/api/profile/create	Create user profile
PUT	/api/profile/update	Update user profile details
DELETE	/api/profile/delete	Delete user profile

Recommendation APIs

Method	Endpoint	Description
POST	/api/recommendation/generate	Generate career recommendations
GET	/api/recommendation/results	Retrieve recommendation results
GET	/api/recommendation/skill-gap	Fetch identified skill gaps
GET	/api/recommendation/roadmap	Retrieve personalized learning roadmap

Resume Parsing APIs

Method	Endpoint	Description
POST	/api/resume/upload	Upload resume file
POST	/api/resume/parse	Extract skills and profile data from resume

Assessment & Test APIs

Method	Endpoint	Description
GET	/api/tests	Retrieve available tests
POST	/api/tests/start	Start assessment test
POST	/api/tests/submit	Submit test answers
GET	/api/tests/result	Fetch assessment results
POST	/api/tests/integrity	Track anti-cheating events and warnings

Feedback APIs

Method	Endpoint	Description
POST	/api/feedback/submit	Submit user feedback
GET	/api/feedback/all	Retrieve feedback records

Admin APIs

Method	Endpoint	Description
POST	/api/admin/login	Admin authentication
GET	/api/admin/users	Retrieve all users
GET	/api/admin/reports	View reports and analytics
POST	/api/admin/tests	Manage MCQ test bank
POST	/api/admin/careers	Manage career datasets and roadmaps

AI Engine APIs

Method	Endpoint	Description
POST	/ai/recommend	Execute recommendation logic
POST	/ai/vectorize	Convert profile data into vectors
POST	/ai/score	Calculate similarity and recommendation scores

System Utility APIs

Method	Endpoint	Description
GET	/api/health	Check backend server health
GET	/api/status	Retrieve system status

Purpose of APIs

These APIs ensure:

- To ensure secure data exchange among system modules
- To efficiently recommend the necessary actions
- To perform real-time processing of the user data
- To ensure seamless interaction between frontend and backend
- To provide modularity and scalability allowing future modifications.